

K-Means Clustering with CUDA

Accelerator-Centric Computing Ecosystems – Programming Assignment Report

Aayush Joglekar
2838702

Aditya Marada
2844807

Nishanthi Srinivasan
2845743

Abstract

K-Means is a popular clustering algorithm that groups data points into K clusters. In this report, we explain how we used CUDA to make a faster version of K-Means by running it in parallel on the GPU. Our approach uses shared memory to speed up access to centroids, warp-level parallelism to quickly calculate distances, and atomic operations to safely update values at the same time. With these changes, our GPU version runs about 45 times faster than the normal CPU version. We also describe how we wrote the code, what choices we made, and how we tested and compared the performance with the regular CPU version.

1 Introduction

K-Means clustering is a fundamental algorithm in data analysis, grouping m data points in an n -dimensional space into K predefined clusters. The algorithm is iterative. We assign each point to the cluster with the nearest centroid based on Euclidean distance and then calculate each cluster's centroid as the mean of its assigned points. We repeat this until convergence criteria, such as minimal centroid movement, minimal point reassignments, or a maximum iteration count, are met.

The iterative nature and data-parallel computations within K-Means i.e., the distance calculations and centroid updates involving potentially millions of points, make it an ideal algorithm for GPU computation. With GPUs we can perform these calculations in parallel across thousands of threads offering significant speedups over the sequential CPU implementation, especially for large datasets.

In this report, we present our CUDA parallel K-Means implementation. Section 2 details the parallelization strategy, including workload distribution and optimization techniques. Section 3 outlines the planned experimental setup and the performance metrics used for evaluation. Finally, in Section 4 will discuss the results, challenges encountered, our findings, and possibilities of future work.

2 Parallelization Approach

The sequential K-Means algorithm, as provided in the file `KMEANS_seq.c`, involved two major computational steps within its iterative loop which we thought were worthy of parallelisation: (1) assigning each point to the nearest centroid and (2) recalculating new centroid positions based on the assignments. Both steps can be parallelised for execution on GPUs.

2.1 Parallelizable Sections

The following parts were identified for parallelisation:

- **Step 1: Point to Centroid Distance Calculation and Assignment**
Calculating the Euclidean distance between each point and

all K centroids, finding the minimum distance, and assigning the point to the corresponding cluster can be done independently for each point. This is implemented in the `kernel_assign_clusters` CUDA kernel.

- **Centroid Recalculation:** This step involves summing the coordinates of all points belonging to each cluster and dividing by the number of points in that cluster. This can be parallelized in two stages:
 - **Accumulation (Step 2):** Summing the coordinates of points within each cluster. This is parallelized across points in the `kernel_accumulate_centroids` kernel using atomic operations.
 - **Finalization (Step 3):** Dividing the accumulated sums by the point count for each cluster to get the new centroid coordinates. This is parallelized across centroids in the `kernel_finalize_centroids` kernel.

Since the convergence check The convergence check - comparing old and new centroids and checking number of changes - is performed on the CPU after each iteration, as it requires global information and synchronization which we believed would not make gain us a lot of improvement on performance if performed on the GPU.

2.2 Workload Distribution (Geometry)

We distribute the workload across the GPU hierarchy (threads, warps, blocks, grid) as follows:

- **Point Assignment ('kernel_assign_clusters'):** The workload is distributed at the granularity of *warps*. Each warp (group of 32 threads) is responsible for processing a single data point X_i . We calculate the distance from this point to all K centroids and determine the nearest one. The grid dimension 'GRID_POINTS' is calculated so that the total number of warps is no less than the number of data points n : 'GRID_POINTS' = $(npoints + WARPS_PER_BLOCK - 1) / WARPS_PER_BLOCK$. Each block contains 'BLOCK_SIZE' threads (e.g., 256), forming 'WARPS_PER_BLOCK = BLOCK_SIZE / WARP_SIZE' warps (e.g., 8). Each block contributes 8 independent warps to the overall workload.
- **Centroid Summation ('kernel_accumulate_centroids'):** The workload is distributed at the granularity of *threads*. Each thread is responsible for processing a single data point (X_i). It reads the point's data and its assigned cluster ID, then atomically adds the point's coordinates to the appropriate global accumulator for that cluster's centroid sum. We reuse the 'GRID_POINTS' grid dimension and 'BLOCK_SIZE' block dimension, mapping each thread to operate on one point.

- **Centroid Finalization ('kernel_finalize_centroids'):** Again, the workload is distributed at the granularity of *threads*. Each thread is responsible for finalizing a single centroid (ce_j). It reads the accumulated sum and the count for its assigned centroid and performs the division for all dimensions. The grid dimension ('GRID_CENTROIDS') is calculated to launch enough blocks such that the total number of threads covers all K centroids: 'GRID_CENTROIDS = $(K + \text{BLOCK_SIZE} - 1) / \text{BLOCK_SIZE}$ '. As this formula may result in some extra threads being launched, threads with $\text{cid} \geq K$ exit immediately to avoid redundant computation.

2.3 Optimizations

We used a few important optimizations to speed up our implementation:

- **Warp-Level Distance Calculation:** In 'kernel_assign_clusters', each warp handles the distance between one point and one centroid. Instead of one thread doing all the work, each thread in the warp works on a part of the dimensions. Threads use strided access to go through their assigned dimensions, calculate squared differences, and then we sum up all these values using '`__shfl_down_sync`'. This gives the total squared distance in lane 0 of the warp. This method makes better use of warp threads, lowers register usage, and speeds up the calculation by using fast communication between threads in a warp.
- **Shared Memory Tiling ('kernel_assign_clusters'):** Reading all K centroids from global memory is slow, so we use tiling with shared memory to make it faster. We divide centroids into smaller chunks (tiles), and each tile is loaded into shared memory by the first few threads in the block. After loading, all warps in the block use this fast shared memory to access the centroids. This cuts down on the number of global memory accesses and makes things faster. We use '`__syncthreads()`' before and after each tile to make sure all threads are synced.

3 Evaluation

3.1 Experimental setup:

3.1.1 Hardware Configuration. The experiments were conducted on a machine with the following hardware specifications:

- CPU: Intel Xeon E5-2630 v3 @ 2.40GHz
- GPU: NVIDIA GeForce GTX TITAN X
- RAM: 128 GB DDR4

3.2 Software Environment

The implementation and experiments were carried out using the following software stack:

- CUDA Toolkit: Version 12.3
- Compiler: nvcc from CUDA 12.3, with -O3 optimization flag
- Programming Language: C++17 for both the sequential and CUDA implementations

3.3 Performance Metrics

The performance of the algorithm was evaluated using the following metrics:

- Execution Time
- Speedup
- Scalability in N (number of data points)
- Scalability in D (dimensionality)
- Scalability in K (number of clusters)
- Kernel-level Runtime Breakdown
- Optimization Payoff

3.4 Experiments:

3.4.1 Speedup Analysis with Dataset Size: This evaluation was performed using datasets of varying sizes: Small ($N = 200$), Medium ($N \approx 102,400$) and Large ($N \approx 358,400$).

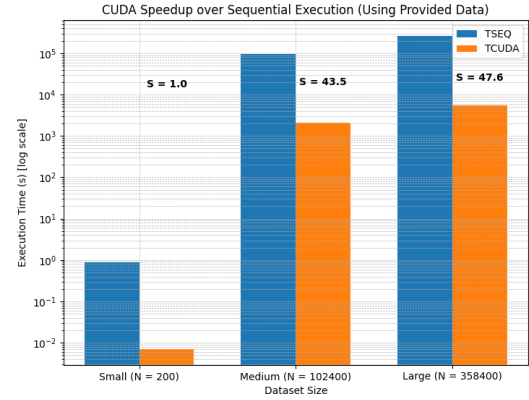


Figure 1: Execution times of the sequential and CUDA implementations on three datasets. Speedup is annotated above CUDA bars.

- (1) **Small dataset:** The speedup is very low (around 1.0x) because the GPU takes time to start kernels and move data. For small data, this setup time takes longer than the actual computation, so the GPU doesn't really help much. GPUs work best when there's a lot of work to do, which isn't the case here.
- (2) **Medium Dataset:** Speedup jumps to 43.5x because the GPU can now use more threads and resources. The overhead gets spread out across more data, so we get much better performance compared to the CPU version.
- (3) **Large Dataset:** The best speedup is 47.6x, but the gain starts to slow down. This means the GPU is reaching its limits. Things like memory speed and scheduling start to matter more, and some parts of the code that can't be parallelized start holding back the performance.

These results demonstrate that GPU acceleration significantly boosts K-means performance, especially for larger datasets.

3.4.2 Scalability Analysis: Runtime vs Problem Size (Fig.2). To evaluate the scalability of the system, we analyze how total runtime

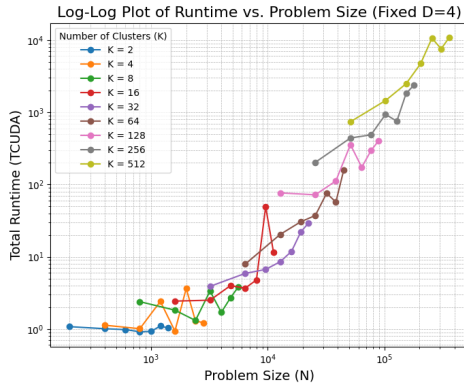


Figure 2: Scalability Analysis of CUDA Runtime

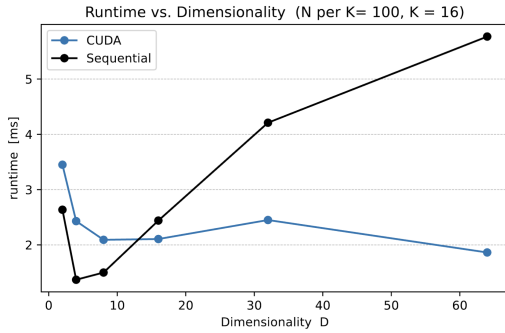


Figure 3: CUDA Runtime vs Dimensions

varies with increasing problem size (N) under a fixed dimension ($D = 4$).

- (1) **Scaling Behavior with Problem Size (N):** Runtime (TCUDA) increases with dataset size for all values of K , with scaling behavior varying. For smaller K values, the runtime shows near-linear or sublinear growth. Higher K values exhibit steeper, more linear curves, indicating superlinear scaling.
- (2) **Effects of Cluster Count (K):** As K increases, runtime rises significantly across all N values. This reflects the added computational load from managing more clusters, including higher memory access and centroid calculations especially when N exceeds 10^4 , where the increasing overhead of managing more clusters becomes evident.
- (3) **Slope Behavior:** For small K , curves flatten as N grows, showing sublinear or linear scaling. However, for large K (e.g., $K = 128$ to 512), steeper lines indicate superlinear growth, suggesting that runtime scaling is dominated by cluster assignments and memory transactions, limiting efficiency for high K values.

3.4.3 Scalability Analysis: Runtime vs Dimensions (Fig. 3).

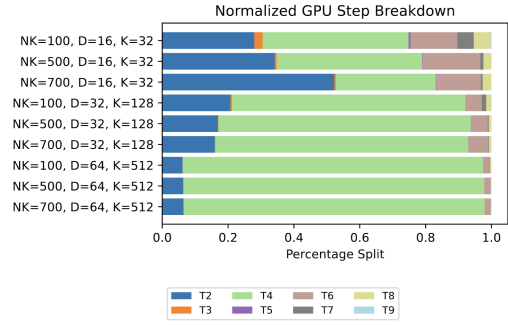


Figure 4: Runtime breakdown of steps

- (1) **CUDA Stability:** The CUDA runtime remains fairly stable as dimensionality increases. After an initial drop, the runtime stays between 2 to 2.5 ms for most values of D , with a slight decrease at $D = 64$. This shows that the GPU handles higher dimensions efficiently without a big increase in cost.
- (2) **Comparison with Sequential:** The gap between CUDA and sequential grows wider as D increases, highlighting the scalability advantage of the GPU implementation. While both versions start close in performance at low dimensions, CUDA clearly outperforms as the feature size increases.

3.4.4 *Runtime breakdown (Fig. 4).* To understand how time is spent inside the CUDA implementation, we break down the execution into key GPU steps (T2–T9) and observe their relative proportions across different values of N , D , and K .

- (1) **Dominance of Kernel Execution (T4):** As expected, the main workload comes from kernel execution (T4). This step takes up most of the time, especially when K and D are large, since it handles point-to-cluster assignments in parallel. This is a good sign—our GPU is working on the hardest part.
- (2) **Transfer Overheads Shrink with Scale:** Initial data transfer (T2) and copying centroids to GPU (T3) take a bigger slice when N is small, but their impact shrinks as the problem size grows. This shows that communication costs don't scale linearly and are amortized over bigger workloads.
- (3) **Accumulate Kernel (T6) Grows with Complexity:** T6, which handles centroid updates, grows in relative weight as K increases. This makes sense—more clusters mean more reduction work. Final steps like T7 (finalize), T8 (copy results), and T9 (check convergence) stay relatively small, which means the rest of the pipeline is efficient.

Kernel	16D	32D	64D
assign_clusters	79.56	99.71	99.78
finalize_centroids	54.91	51.51	36.00
accumulate_centroids	100.00	100.00	100.00

Table 1: Warp Execution Efficiency of Kernels at Different Dimensions

3.4.5 *Profiling of GPU with nvprof.* Table 1 shows that the `assign_clusters` kernel gets more efficient as the number of dimensions goes up. It starts at about 79% for 16D and reaches almost 100% at 32D and 64D. This makes sense because one warp handles 32 dimensions at once. So, if D is less than 32, some threads don't do anything. But once D hits 32 or more, all threads are used properly. The `accumulate_centroids` kernel always runs at 100% efficiency, which means the work is spread out well. On the other hand, the `finalize_centroids` kernel gets worse as D increases. This might be because the work isn't split evenly or some threads are just waiting around.

Kernel	16D	32D	64D
<code>assign_clusters</code>	0.867	0.867	0.868
<code>finalize_centroids</code>	0.054	0.052	0.063
<code>accumulate_centroids</code>	0.911	0.920	0.959

Table 2: Achieved Occupancy of Kernels at Different Dimensions

Table 2 shows how well each kernel used the GPU threads at different dimensions. The `assign_clusters` kernel did well and stayed around 0.87 occupancy for all dimensions, which means it used the threads efficiently. The `accumulate_centroids` kernel did even better, going up slightly with more dimensions and reaching 0.96 at 64D. But the `finalize_centroids` kernel had really low occupancy the whole time, always under 0.07. This means it didn't use the GPU threads properly, maybe because it had less work to do or didn't access memory in a good way.

4 Discussion and Conclusion

Our CUDA-based implementation of K-Means clustering significantly improves performance over the sequential CPU version, especially as dataset sizes grow. We observed speedups of 43.5x for a medium-sized dataset ($N \approx 102,400$) and 47.6x for a large dataset ($N \approx 358,400$). These results highlight the effectiveness of GPU parallelism for compute-intensive, data-parallel problems like K-Means.

4.1 Challenges and Limitations:

Despite significant speedups, several challenges limited the full potential of the CUDA implementation.

A key bottleneck was atomic contention during the centroid accumulation phase. With many threads attempting to update shared memory locations simultaneously, memory updates became serialized, degrading performance—especially at high values of K or with imbalanced cluster distributions.

Another limitation was the CPU-side convergence check, which introduced a synchronization barrier at the end of each iteration. This CPU-GPU communication overhead increased latency and partially offset the benefits of GPU parallelism. Offloading this step to the device could further improve performance.

Hardware-related constraints also emerged, particularly around global memory bandwidth. As dataset sizes grew, memory transfer between global memory and compute cores became a limiting

factor. Additionally, non-coalesced memory accesses caused by irregular access patterns further reduced throughput, especially during centroid updates that were not aligned across thread blocks.

4.2 Future Work

While current optimizations, including warp-level distance calculation and shared memory tiling, have significantly improved performance, several additional techniques could push the boundaries of efficiency further.

- **Hierarchical Reductions:** Performing accumulation in multiple stages within warps, then blocks, and finally across blocks can reduce global atomic operations and mitigate contention during centroid updates.
- **Cooperative Groups:** Leveraging CUDA cooperative groups could enable more efficient thread synchronization at finer granularity. This would allow device-side convergence checks and large-scale reductions without costly CPU-GPU communication.
- **Memory Access Optimization:** Structuring data to improve memory coalescing, ensuring threads access contiguous memory—can reduce latency and increase throughput, especially during frequent centroid accesses.

4.3 Summary of Findings

- **GPU acceleration offers massive speedups** for K-Means when dataset size is large enough to offset kernel launch and memory transfer overheads.
- **Warp-level parallelism and shared memory tiling** are effective for optimizing distance calculations and centroid access.
- **Atomic operations and memory bandwidth** become critical bottlenecks, especially with larger cluster counts.
- **Sequential CPU components**, such as convergence checking, limit scaling due to Amdahl's Law, which restricts speedup based on serial execution time.